Project 1: MTUHash Function Implementation

Total: 100 points
DDL: 11:59 PM, 03/13/2026
There will be no extension, please do it as early as possible.
The project description may be continuously refined based on feedback from students. Please use the latest version.

This is the only project that requires programming. The project 2 and project 3 will not be programming-oriented.

You can use the following language to finish the project: 1) Linux C/C++ (preferable), 2) Python, or 3) Java. The following project description is based on Linux C/C++, the corresponding description using Python and Java are similar (and thus skipped).

FAQ: "**How to compile C/C++ under Linux?**"

Answer: you are suggested to use gcc to compile C/C++ code. A short tutorial can be found at https://bit.ly/3xAfIZc. More tutorials can be found online.

FAQ: "**I am not familiar with programming, what should I do?"**

Answer: you can find partners (who are familiar with programming) to work together.

Collaboration:

- You are allowed to work on this project in a group of at most 3 students. All students in the same team will get the same score. You can also work on the project individually. However, there will be no bonus points for individual work.

Overview. In this project, there are two tasks. (1) You are required to implement a hash function named MTUHash. (2) You are required to verify the avalanche effect (AE) of MTUHash.

Task 1. MTUHash Implementation (70 ponts)

The MTUHash takes a message (to be hashed) as input and outputs a fixed 32-bit value. Note that the output with 32-bit length is not very secure in practice. Here is a simplification of the problem.

We assume that the input message has a fixed length of 32 bits. It is a simplification of the problem. Some examples of 32-bit input messages (stored in txt files) can be found at Canvas.

The overview of MTUHash is shown in Figure 1. It has 16 rounds of operations.

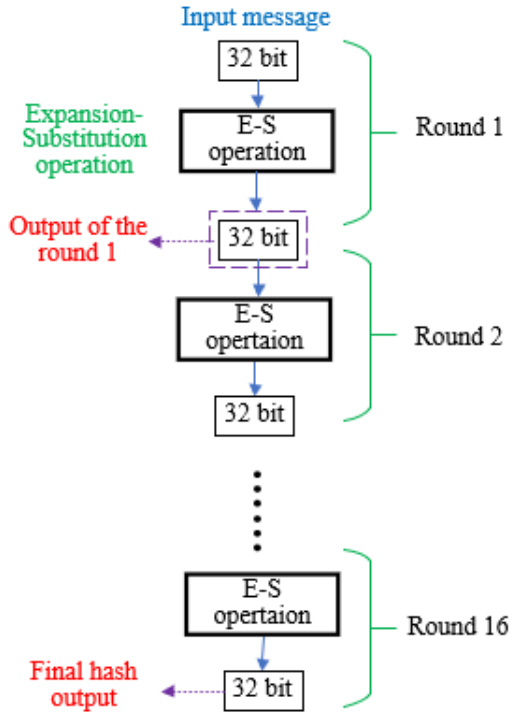
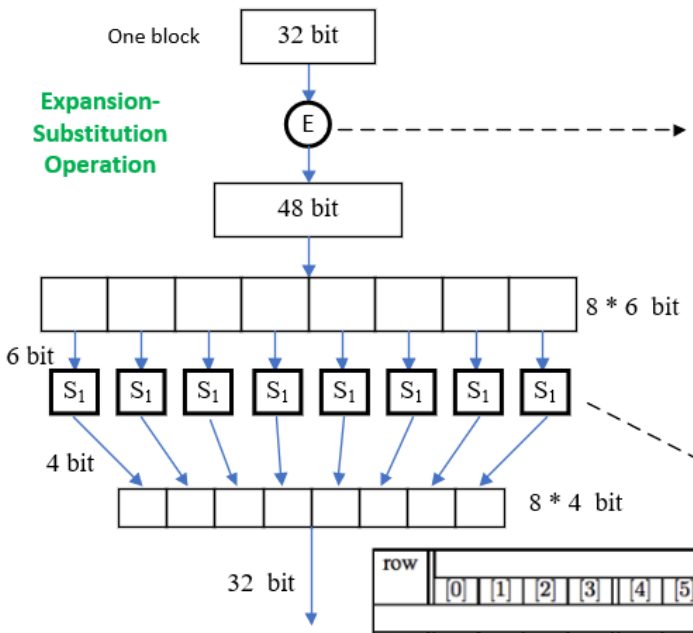


Fig 1. An overview of MTUHash.



E					
32	1	2	3	4	5
4	5	6	7	8	9
8	9	10	11	12	13
12	13	14	15	16	17
16	17	18	19	20	21
20	21	22	23	24	25
24	25	26	27	28	29
28	29	30	31	32	1

Expansion Table

row	column number															
	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]	[10]	[11]	[12]	[13]	[14]	[15]
S_1																
[0]	14	4	13	1	2	15	11	8	3	10	6	12	5	9	0	7
[1]	0	15	7	4	14	2	13	1	10	6	12	11	9	5	3	8
[2]	4	1	14	8	13	6	2	11	15	12	9	7	3	10	5	0
[3]	15	12	8	2	4	9	1	7	5	11	3	14	10	0	6	13

Fig 2. E-S operation in MTUHash.

In each round, the 32-bit input has the expansion and substitution operation (E-S operation). The E-S operation is shown in Figure 2.

E-S Operation. Each 32-bit input first has the expansion operation based on the DES's expansion table (as shown in Figure 2). Then, the 48-bit message is divided into 8 pieces of 6-bit blocks. Each 6-bit message is fed to a S-box S_1 and generates a 4-bit output. The S-box S_1 is shown in Figure 2. Subsequently, all of the 4-bit outputs are concatenated together to generate a 32-bit output. The rules used in the S-box are identical to the rules used in DES (please check the course slides). In DES, 8 different S-boxes are used, but MTUHash only uses one uniform S-box (see Figure 2, all 6-bit blocks use an identical S-box) to simplify the programming workload.

Note. MTUHash employs some operations from DES to increase its avalanche effect. Therefore, by finishing this project, you will also be familiar with the basic operations used in DES and some other similar symmetric encryption algorithms.

You are required to write a program named MTUHash.cpp (or MTUHash.c) that implements MTUHash. The program takes the input message (stored in Hashin.txt) as the MTUHash input. We assume that the length of the input message in Hashin.txt is always 32 bits. Then, the program generates two output files. First, the output of the round 1 (see Figure 1) is saved in Out1.txt. Second, the final hash output is saved in OutFinal.txt. Note that all the input and outputs are in the format of bit arrays. Please keep the output in bit array format (the same as the input).

input: Hashin.txt
output: Out1.txt and OutFinal.txt

Note. You may optionally write a logic to test the length of the input. If the input length is not 32 bits, then the program prints "Length Error". In evaluation, the length of the input message will always be 32 bits.

Task 2. Verify the Avalanche Effect (AE) of MTUHash (30 points)

The AE for a hash function is defined as follows.

AE: if the input message flips a single bit, then each bit of the hash output has a nearly 50% probability of being flipped.

You are required to write a program named AECalculator.cpp (or AECalculator.c) that verifies the AE of MTUHash. The program takes the message (stored in Hashin.txt) as the MTUHash input and computes the averaged flipped rate (AFR) of the MTUHash output. Then, the computed AFR value is stored in AFR.txt. The output AFR should be in the format of percentage and please keep

one digit after the decimal points. For example, 51.5%, 49.4%. etc. Note that we assume that the length of the input message in Hashin.txt is always 32 bits.

input: Hashin.txt
output: AFR.txt

More specifically, let m_0 denote the input message (in bit array format). Let l=32 be its total bit length. The program works as follows.

Step 1. The program computes the original hash output MTUHash(m_0) (i.e., the final hash output after 16 rounds). Then, the program flips the 1st bit of m_0 (e.g., 1=>0 or 0=>1) and then computes its MTUHash output again. Next, the program computes how many percentages of bits in the final hash output are flipped. Let P_1 represent the percentage of flipped bits, where the subscript "1" means the 1st bit of m_0 is flipped. For example, assume the original MTUHash(m_0) = 000111. After the 1st bit of m_0 is flipped, then the new hash input becomes 100111. Assume that MTUHash(100111) = 101011. Therefore, we can compute P_1 = 3/6 = 50%. Note that the real MTUHash output is 32 bits, in the above example, the output is 6 bits (it is a simplification).

Step 2. The program flips the 2nd bit of m_0 and computes P_2.

Step 3. Continue until the program computes P_l, where l=32 is the total bit length of m_0.

Step 4. Compute the average value of P_1, …, P_l. The average value is defined as AFR and it is saved in AFR.txt.

Note. You can make the MTUHash (that is implemented in Task 1) as a function, and then call the MTUHash function to finish the second task. After computation, you can check the result in AFR.txt to verify whether it is close to 50% (Note that MTUHash may not satisfy this property since it is an oversimplified design).

Submission: Please submit your final project outcome to Canvas course webpage. The project submission should be put in a folder named "Proj1" and compressed into Proj1.zip. The folder should include:

1) authors.txt: write down the information for the author(s), including full name and student ID. For teamwork, each author's information should be included.

2) Source: a folder contains all necessary source files for compilation and running. Please add some necessary comments (on key steps) to your source code.

3) readme.txt: please describe any necessary steps for the grader to compile, run, and test your code.
4) Results: a folder named "Results". The files that are contained in it are introduced in the Evaluation part on the following pages.

4) For group work, only one submission from one student is OK.

5) Providing Makefile is preferred, but not mandatory.

6) Using C++ as an example, in your submission, please use the directory structure as shown below.

Please keep the files clean and remove all files generated after compilation.

```
Proj1
   |---authors.txt
   |---readme.txt
   |-- Results
   |---Source
       |--- MTUHash.cpp
       |--- AECalculator.cpp
       |--- Hashin.txt
       |--- …
```

Note: if you code using Python or Jave, please follow similar rules: 1) put author information in author.txt, 2) put source codes in Source folder, 3) put evaluation results in Results folder, etc.

Evaluation: The grader will check the program based on the following steps:

Two input files Hashin1.txt and Hashin2.txt will be used in evaluation. The two files can be downloaded from Canvas.

(1) Check the MTUHash by running:
./MTUHash Hashin1.txt

Please re-name the newly generated Out1.tex to Out1-test1.txt

Please rename the newly generated OutFinal.txt to OutFinal-test1.txt
Please copy Out1-test1.txt and OutFinal-test1.txt to Results folder.

(2) Check the MTUHash by running:
./MTUHash Hashin2.txt

Please re-name the newly generated Out1.tex to Out1-test2.txt
Please rename the newly generated OutFinal.txt to OutFinal-test2.txt
Please copy Out1-test2.txt and OutFinal-test2.txt to Results folder.

(3) Check the AECalculator by running:
./AECalculator Hashin1.txt

Please re-name the newly generated AFR.tex to AFR1.txt
Please copy AFR1.txt to Results folder.

(3) Check the AECalculator by running:
./AECalculator Hashin2.txt

Please re-name the newly generated AFR.tex to AFR2.txt
Please copy AFR2.txt to Results folder.

The grader will check the correctness of all files in Results folder.
If you use Python or Java, please keep the Results folder the same as described above.

Note. If your program makes it difficult for the grader to evaluate (e.g., your program cannot be complied or there are no instructions for how to compile it in readme.txt file), then you may lose 5-15 points according to the degree of difficulty.

Project Description Update. If you find any issues in the project description (e.g., not clear), please contact the instructor. The instructor will continuously update the project description to make it more accurate and clearer.